# Towards One-Step Flow Matching: A Survey of Velocity-Based Methods

**Nhat H. Tran**
Hanoi University of Science and Technology
Hanoi, Vietnam
nhat.th261124m@sis.hust.edu.vn

**Luong T. Nguyen**
Hanoi University of Science and Technology
Hanoi, Vietnam
luong.nt252743m@sis.hust.edu.vn

**Alisa S. Hamidova**
Hanoi University of Science and Technology
Hanoi, Vietnam
alisa.hs252893m@sis.hust.edu.vn

## Abstract

Flow matching has emerged as a powerful simulation-free framework for continuous generative modeling, yet producing a single high-quality sample still requires tens to hundreds of neural function evaluations. This survey traces the rapidly evolving push toward one-step flow matching: the distillation of an entire generative trajectory into a single network call. We begin by reviewing the foundations of flow matching and conditional flow matching, then examine traditional acceleration strategies including cache-based sampling and step distillation. We next discuss closely related paradigms such as consistency models and flow-map distillation, which provide the conceptual stepping stones for direct noise-to-data transport. The core of the survey is devoted to three modern one-step families—consistency flow matching, mean flows, and AlphaFlow—analyzing their training objectives, theoretical guarantees, and practical trade-offs. We further summarize experimental setups, standard baselines, and state-of-the-art results, showing that the quality gap between one-step and many-step generation has narrowed substantially. By unifying these developments under a single narrative, we highlight the field's progression from simulation-based sampling to direct prediction of the transport map, and provide practical guidance for researchers entering efficient generative modeling.

## 1 Introduction

Generative modeling has undergone a profound transformation with the advent of diffusion models and their continuous-time counterparts, flow-based models [Ho et al., 2020, Song et al., 2021, Lipman et al., 2023]. These approaches construct a transport between a simple noise distribution and a complex data distribution by learning a time-dependent velocity field that governs the evolution of probability mass through an ordinary differential equation (ODE). While diffusion models have achieved state-of-the-art results in image synthesis, audio generation, and scientific simulation, they share a critical limitation: generating a single sample requires simulating the ODE through many time steps, each demanding a forward pass of a large neural network. For state-of-the-art Flow Matching models, this can translate to $50$–$1000$ function evaluations (NFEs), rendering real-time or interactive deployment impractical.

Flow Matching (FM) and its most common variant, Conditional Flow Matching (CFM) [Lipman et al., 2023], provide an especially elegant formulation of this family. By regressing vector fields of fixed conditional probability paths, FM offers a simulation-free training objective for continuous

Figure 1: One-step samples from AlphaFlow on ImageNet (illustrative; replace with actual AlphaFlow outputs) [Zhang et al., 2025].

normalizing flows and yields straight, optimal-transport trajectories that are faster to integrate than the curved paths typical of diffusion models. Beyond images, FM and CFM have found broad application in speech synthesis, where they power neural vocoders and text-to-speech systems by mapping latent noise directly to raw waveforms or mel-spectrogram features; they are also used in music generation, video synthesis, protein design, and molecule generation. These successes make flow matching a natural and timely lens through which to study efficient generative modeling.

This computational bottleneck has catalyzed extensive research toward one-step and few-step generative models. The central question is whether it is possible to distill the expensive many-step sampling process into a direct map that transports noise to data in a single neural network evaluation, while preserving the quality, diversity, and controllability of the original model. Early approaches to this problem, such as caching intermediate computations, progressive distillation [Salimans and Ho, 2022], and consistency models [Song et al., 2023], provided proof-of-concept results but struggled with scalability, multi-step degradation, or the need for pre-trained teachers.

More recently, flow matching has provided a unified lens for understanding these acceleration efforts. It frames generative modeling as learning a probability flow ODE, and one-step methods can be seen as different strategies for bypassing the costly simulation of that ODE—whether by enforcing consistency along trajectories, predicting average velocities, or matching terminal dynamics. Despite their differing objectives, these approaches share a common aim: replacing iterative sampling with direct, single-step or few-steps prediction.

This survey traces the development of one-step flow matching from its conceptual origins to the current state of the art. Our contributions are as follows:

- We provide a unified narrative that connects flow matching, traditional acceleration techniques, consistency models, and modern distillation methods within a single mathematical framework.
- We analyze three key methodological advances—consistency flow matching, mean flows, and AlphaFlow—comparing their training requirements, theoretical guarantees, and empirical performance.
- We summarize experimental setups and standard baselines used to evaluate one-step models, placing state-of-the-art results in a unified comparative context.
- We highlight practical trade-offs between training from scratch and distilling from a teacher, and identify open challenges for real-world deployment.

The remainder of this survey is organized as follows. Section 2 reviews related work, including the foundations of flow matching and conditional flow matching (section 2.1), traditional acceleration strategies such as caching and step distillation (section 2.2), and related paradigms including consistency models (section 2.3). Section 3 presents the three core families of one-step methods:

**Algorithm 1** Flow Matching Training

**Require:** Data $x_1$, noise $x_0$, velocity network $v_\theta$, optimizer

Sample $t \sim \text{Uniform}(0, 1)$
$x_t \leftarrow (1-t)\, x_0 + t\, x_1$
$u_t \leftarrow x_1 - x_0$ ▷ Conditional velocity for OT path
$v_{\text{pred}} \leftarrow v_\theta(x_t, t)$
$\mathcal{L} \leftarrow \|v_{\text{pred}} - u_t\|^2$
Update $\theta$ with $\nabla_\theta \mathcal{L}$
**return** $\mathcal{L}$

consistency flow matching, mean flows, and AlphaFlow. Section 4 describes the experimental setup, datasets, metrics, and baselines. Section 5 synthesizes the empirical results. Section 7 concludes with future directions.

## 2 Related Work

Flow matching provides the mathematical foundation for the one-step methods surveyed in this paper. In this section, we review the core ideas of flow matching and conditional flow matching, trace traditional strategies for accelerating diffusion and flow models, and situate the modern one-step literature among closely related paradigms.

### 2.1 Flow Matching and Conditional Flow Matching

Flow Matching (FM) [Lipman et al., 2023] introduces a simulation-free paradigm for training Continuous Normalizing Flows (CNFs). The key idea is to learn a time-dependent velocity field $v_\theta(t, x)$ whose associated probability flow ODE transports a simple source distribution $p_0$ (typically Gaussian noise) to a target data distribution $p_1$:

$$\frac{d}{dt} x_t = v_\theta(t, x_t), \qquad x_0 \sim p_0, \quad x_1 \sim p_1. \tag{1}$$

Rather than simulating this ODE during training, FM defines a conditional path $x_t = \alpha_t x_0 + \beta_t x_1$ between each data-noise pair $(x_0, x_1)$ and regresses the network prediction against the corresponding conditional velocity $u_t = \dot{\alpha}_t x_0 + \dot{\beta}_t x_1$.

The marginal velocity field is the expectation of $u_t$ over all $(x_0, x_1)$ pairs consistent with $x_t$, which is intractable to compute directly. Conditional Flow Matching (CFM) sidesteps this difficulty by using the conditional objective

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, x_0, x_1} \|v_\theta(x_t, t) - u_t\|^2, \tag{2}$$

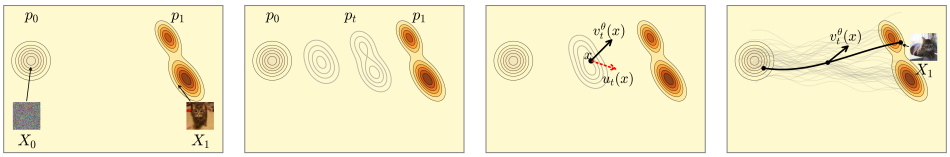
which shares the same gradient with respect to $\theta$ as the intractable marginal objective while remaining computationally efficient [Lipman et al., 2023]. The minimizer of eq. (2) equals the true marginal velocity field, so training amounts to standard regression on sampled $(t, x_0, x_1)$ tuples.

A particularly important choice is the *optimal transport (OT) path*, where $\alpha_t = 1 - t$ and $\beta_t = t$. This yields the straight-line interpolant $x_t = (1-t)x_0 + tx_1$ and a constant conditional velocity $u_t = x_1 - x_0$. These straight paths minimize transport cost, enable faster training, and have been shown to improve generalization and sample quality compared with the curved trajectories typical of diffusion models [Lipman et al., 2023].

Flow matching thus provides the training backbone for all subsequent methods in this survey: it defines the probability flow ODE that one-step methods seek to compress, and the conditional regression objective that underlies both standard and one-step training.

### 2.2 Traditional Methods

Before the emergence of dedicated one-step flow matching, practitioners accelerated generative models through two broad strategies: reducing the cost of each function evaluation via caching, and reducing the number of evaluations via step distillation. Both approaches predate modern one-step FM but remain relevant as baselines and complementary techniques.




(a) Data. (b) Path design. (c) Training. (d) Sampling.

Figure 2: The Flow Matching blueprint. (a) The goal is to find a flow mapping samples $X_0$ from a known source or noise distribution $p$ into samples $X_1$ from an unknown target or data distribution $q$. (b) To do so, design a time-continuous probability path $(p_t)_{0\leq t\leq 1}$ interpolating between $p := p_0$ and $q := p_1$. (c) During training, use regression to estimate the velocity field $u_t$ known to generate $p_t$. (d) To draw a novel target sample $X_1 \sim q$, integrate the estimated velocity field $u_t^\theta(X_t)$ from $t = 0$ to $t = 1$, where $X_0 \sim p$ is a novel source sample.

### 2.2.1 Cache-based acceleration

Because Conditional Flow Matching is typically sampled with many-step ODE solvers, its inference pipeline resembles that of a diffusion model: a deep network is evaluated sequentially, and adjacent evaluations often produce highly similar intermediate activations. Consequently, flow matching can directly adopt training-free caching methods originally developed for diffusion.

Formally, one Euler step of a flow-matching sampler can be written as

$$x_{t+\Delta t} = x_t + \Delta t\, v_\theta(x_t, t), \tag{3}$$

where $v_\theta(x_t, t)$ is the neural velocity estimate. If we decompose the network into a feature extractor $\phi_\theta$ and a read-out head $g_\theta$, so that $v_\theta(x_t, t) = g_\theta\big(\phi_\theta(x_t, t)\big)$, then caching stores the features $h_t = \phi_\theta(x_t, t)$ from a previous step and reuses them when the timestep changes slowly:

$$\tilde{v}_\theta(x_t, t) = g_\theta\big(h_{t-\delta}\big). \tag{4}$$

DeepCache [Ma et al., 2023] is the canonical example: it caches high-level U-Net features across denoising steps and updates only low-level features, yielding large speedups on Stable Diffusion and latent diffusion models with minimal quality loss. More recent diffusion caching schemes such as FasterCache [Lv et al., 2024] and TeaCache [Liu et al., 2025] refine this idea for video models by exploiting redundancy in conditional/unconditional branches and timestep-aware feature reuse, respectively. Because these methods operate on the backbone architecture rather than the sampling distribution, they can be ported to flow-matching image and video generators with little or no modification.

Work that explicitly targets flow matching caching is still scarce. MeanCache [Gao et al., 2026] is a recent exception: it argues that instantaneous-velocity feature caching introduces trajectory drift, and instead caches Jacobian–vector products to construct interval average velocities for more stable FM acceleration. Outside of vision, predictive feature caching has been applied to flow-matching models of molecular geometry generation [Sommer et al., 2025], and layer-selective attention caching has been used to accelerate flow-matching transformers for audio separation. These domain-specific works confirm that caching remains a valuable, largely model-agnostic acceleration tactic. Nevertheless, caching lowers the cost per step rather than the step count, so it is complementary to—rather than a replacement for—the one-step methods surveyed below.

### 2.2.2 Step distillation

Step distillation trains a student model that requires fewer sampling steps than its teacher. The prototypical approach is progressive distillation [Salimans and Ho, 2022], which iteratively halves the number of sampling steps by distilling a $2N$-step teacher into an $N$-step student. Given a teacher sampler $f_{\theta^*}(\cdot, \cdot; 2N)$ that produces an estimate after $2N$ function evaluations, progressive distillation minimizes

$$\mathcal{L}_{\text{PD}}(\theta) = \mathbb{E}_{t,x_t}\Big[\big\| f_\theta(x_t, t; N) - f_{\theta^*}(x_t, t; 2N)\big\|^2\Big], \tag{5}$$

where $f_\theta(x_t, t; N)$ denotes the student output obtained with $N$ steps. Although effective for moderate speedups, progressive distillation is expensive—the distillation procedure must be repeated for each halving—and can suffer from error accumulation as the step budget shrinks.

A closely related idea is rectified flow [Liu et al., 2022], which learns straight ODE trajectories by refitting the transport map between noise and data. It can be viewed as minimizing the same conditional objective as FM but with the goal of straightening the coupling:

$$\mathcal{L}_{\text{RF}}(\theta) = \mathbb{E}_{t,x_0,x_1}\Big[\big\|v_\theta\big((1-t)x_0 + tx_1, t\big) - (x_1 - x_0)\big\|^2\Big]. \tag{6}$$

Repeated rectification yields increasingly straight flows, and when the flow becomes exactly straight a single Euler step gives exact transport. InstaFlow [Liu et al., 2024] combined rectified flow with distillation to produce the first Stable-Diffusion-quality one-step text-to-image model, demonstrating that flow-based distillation could outperform progressive distillation on standard benchmarks.

These methods established the feasibility of few-step and one-step sampling, but they also revealed key limitations—expensive multi-stage training, dependence on high-quality teachers, and difficulty in scaling to high resolutions—that motivated the dedicated one-step flow matching methods discussed in section 3.

### 2.3 Related Methods

Two closely related lines of work set the stage for modern one-step flow matching: consistency models, which enforce that all points on a trajectory map to the same endpoint, and flow-map distillation, which learns the general transport operator between arbitrary times.

#### 2.3.1 Consistency Models

Consistency Models (CMs) [Song et al., 2023] train a network $f_\theta(x_t, t)$ that maps any point on a diffusion or flow trajectory directly to the trajectory's endpoint $x_0$. The defining consistency property requires that $f_\theta(x_t, t) = f_\theta(x_{t'}, t')$ whenever $x_t$ and $x_{t'}$ lie on the same trajectory. CMs demonstrated that one-step generation was possible without iterative sampling, but early variants struggled with scalability, multi-step error accumulation, and the need for careful curriculum training. These limitations motivated the consistency-flow-matching approaches discussed in section 3.

#### 2.3.2 Flow-map distillation (out of scope)

A related but distinct direction is flow-map distillation, which generalizes the consistency-model idea from endpoint prediction to learning the *flow map* $f_\theta(x_t, t, s)$ that transports a state from time $t$ to any other time $s$ along the teacher ODE. Methods such as Align Your Flow [Sabour et al., 2025] have scaled this idea to high-resolution image generation. Because flow-map methods operate on a different design axis than the velocity-based methods that are the focus of this survey, we do not discuss them in detail; we note them here for completeness and refer interested readers to the original works.

## 3 One-Step Generative Modeling

Building on the flow matching framework, recent methods have developed increasingly sophisticated techniques for learning one-step or few-step generators. We discuss three representative velocity-based approaches that span the spectrum from training from scratch to distillation from pre-trained models. Flow-map distillation methods are related but outside the scope of this survey; we noted them briefly in section 2.3.

### 3.1 Consistency Flow Matching

Consistency Flow Matching (Consistency-FM) [Zhou et al., 2024] proposes to enforce *velocity consistency* along flow trajectories, directly defining straight flows without requiring optimal transport coupling or expensive transport plans.

Given a flow trajectory $\gamma_x(t)$ with velocity $v(t, \gamma_x(t))$, the consistency condition requires the velocity to be constant along the trajectory:

$$v(t, \gamma_x(t)) = v(s, \gamma_x(s)) \quad \forall t, s \in [0, 1]. \tag{7}$$

**Algorithm 2** Consistency Flow Matching Training

**Require:** Data $x_1$, noise $x_0$, velocity network $v_\theta$, EMA model $v_{\theta^-}$, optimizer, hyperparameter $\alpha$

Sample $t \sim \text{Uniform}(0, 1 - \Delta t)$
$x_t \leftarrow (1-t)\, x_0 + t\, x_1$
$x_{t+\Delta t} \leftarrow (1-(t+\Delta t))\, x_0 + (t+\Delta t)\, x_1$
$v_t \leftarrow v_\theta(x_t, t)$
$v_{t+\Delta t} \leftarrow v_\theta(x_{t+\Delta t}, t+\Delta t)$
$f_t \leftarrow x_t + (1-t)\, v_t$
$f_{t+\Delta t} \leftarrow x_{t+\Delta t} + (1-(t+\Delta t))\, v_{t+\Delta t}$
▷ with no grad
$f^{\text{ema}}_{t+\Delta t} \leftarrow x_{t+\Delta t} + (1-(t+\Delta t))\, v_{\theta^-}(x_{t+\Delta t}, t+\Delta t)$
$f^{\text{ema}}_t \leftarrow x_t + (1-t)\, v_{\theta^-}(x_t, t)$
$\mathcal{L}_f \leftarrow \|f_t - f^{\text{ema}}_{t+\Delta t}\|^2 + \|f_{t+\Delta t} - f^{\text{ema}}_t\|^2$
$\mathcal{L}_v \leftarrow \|v_t - v_{\theta^-}(x_{t+\Delta t}, t+\Delta t)\|^2 + \|v_{t+\Delta t} - v_{\theta^-}(x_t, t)\|^2$
$\mathcal{L} \leftarrow \mathcal{L}_f + \alpha\, \mathcal{L}_v$
Update $\theta$ with $\nabla_\theta \mathcal{L}$; update EMA model
**return** $\mathcal{L}$

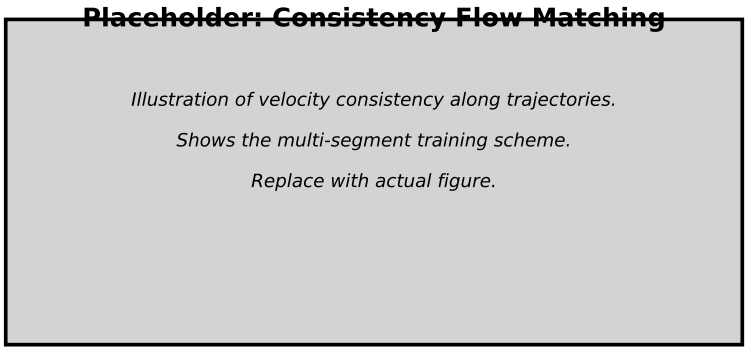



Figure 3: Placeholder: Illustration of Consistency Flow Matching velocity consistency. Replace with a figure showing the multi-segment training scheme.

This is equivalent to the trajectory condition $\gamma_x(t) + (1-t)v(t, \gamma_x(t)) = \gamma_x(s) + (1-s)v(s, \gamma_x(s))$, which implies that the quantity $f(t, x_t) = x_t + (1-t)v_\theta(t, x_t)$ is constant along the path. The training objective is:

$$\mathcal{L}_\theta = \mathbb{E}_{t, x_t, x_{t+\Delta t}} \Big[ \|f_\theta(t, x_t) - f_{\theta^-}(t+\Delta t, x_{t+\Delta t})\|^2 + \alpha \|v_\theta(t, x_t) - v_{\theta^-}(t+\Delta t, x_{t+\Delta t})\|^2 \Big], \quad (8)$$

where $\theta^-$ denotes an exponentially moving average of the parameters. The first term enforces consistency in the endpoint prediction, while the second term regularizes the velocity field.

For complex distributions where a single straight path is insufficient, Consistency-FM introduces *multi-segment training*, which divides the time interval $[0, 1]$ into $K$ segments and learns a consistent velocity $v^i_\theta$ within each segment. This allows piece-wise linear trajectories that can better approximate curved transport paths.

### 3.2 Mean Flows

Mean Flow [Geng et al., 2025] addresses a fundamental limitation of previous methods: the reliance on pre-trained teacher models or expensive distillation pipelines. It proposes a principled framework for one-step generative modeling that can be trained from scratch.

**Algorithm 3** MeanFlow Training

**Require:** Data $x$, noise $\epsilon$, average-velocity network $u_\theta$, optimizer

Sample $r, t \sim \text{Uniform}(0, 1)$ with $r < t$

$z_t \leftarrow (1-t)\,x + t\,\epsilon$ ▷ Interpolant

$v \leftarrow \epsilon - x$ ▷ Instantaneous conditional velocity

$u_{\text{pred}} \leftarrow u_\theta(z_t, r, t)$

Compute JVP: $(v, 1) \cdot \nabla u_\theta(z_t, r, t)$ ▷ Total derivative $\frac{d}{dt}u$

$u_{\text{tgt}} \leftarrow v - (t-r) \cdot \frac{d}{dt}u$ ▷ MeanFlow Identity

$\mathcal{L} \leftarrow \|u_{\text{pred}} - \text{sg}(u_{\text{tgt}})\|^2$

Update $\theta$ with $\nabla_\theta \mathcal{L}$

**return** $\mathcal{L}$

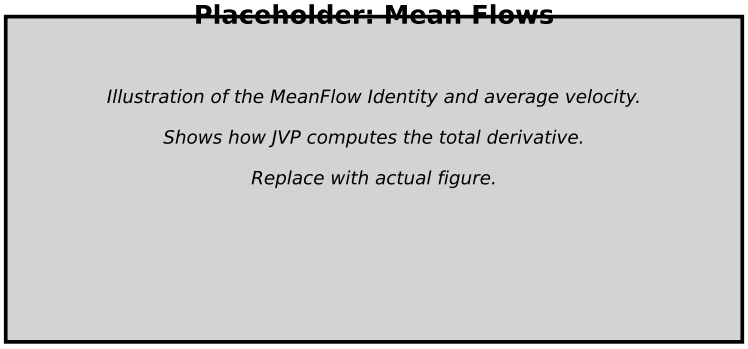



Figure 4: Placeholder: Illustration of the MeanFlow Identity and average velocity. Replace with a figure showing how JVP computes the total derivative.

The key insight is to model the *average velocity* rather than the instantaneous velocity. For a trajectory $z_t$, the average velocity between times $r$ and $t$ is defined as:

$$u(z_t, r, t) = \frac{1}{t-r} \int_r^t v(z_\tau, \tau)\, d\tau. \tag{9}$$

Differentiating the definition $(t-r)u = \int_r^t v\, d\tau$ with respect to $t$ yields the *MeanFlow Identity*:

$$u(z_t, r, t) = v(z_t, t) - (t-r)\frac{d}{dt}u(z_t, r, t), \tag{10}$$

where the total derivative $\frac{d}{dt}u = v \cdot \partial_z u + \partial_t u$ is computed efficiently as a Jacobian-Vector Product (JVP).

The training objective regresses the network prediction $u_\theta$ against the target derived from the MeanFlow Identity:

$$\mathcal{L}(\theta) = \mathbb{E}\|u_\theta(z_t, r, t) - \text{sg}(v(z_t, t) - (t-r)(v(z_t, t) \cdot \partial_z u_\theta + \partial_t u_\theta))\|^2, \tag{11}$$

where sg denotes the stop-gradient operator. Importantly, MeanFlow trains entirely from scratch without distillation or curriculum learning, and achieves state-of-the-art results: FID 3.43 with a single function evaluation on ImageNet 256×256.

### 3.3 AlphaFlow

AlphaFlow [Zhang et al., 2025][1] addresses a subtle but important limitation of MeanFlow: the MeanFlow objective is actually composed of two competing terms whose gradients conflict during

[1] Official implementation: `https://github.com/snap-research/alphaflow`.

**Algorithm 4** AlphaFlow Training

**Require:** Data $x$, noise $\epsilon$, average-velocity network $u_\theta$, iteration $k$, schedule $\alpha(k)$, optimizer

Sample $r, t \sim \text{Uniform}(0,1)$ with $r < t$
$\alpha \leftarrow \alpha(k)$; $\quad s \leftarrow \alpha r + (1-\alpha)t$
$z_t \leftarrow (1-t)\,x + t\,\epsilon$ ▷ Interpolant
$v \leftarrow \epsilon - x$ ▷ Conditional velocity
**if** $\alpha = 0$ **then**
  $u \leftarrow u_\theta(z_t, r, t)$
  Compute JVP: $\frac{d}{dt} u_\theta(z_t, r, t)$
  $u_{\text{tgt}} \leftarrow v - (t-r) \cdot \frac{d}{dt} u$ ▷ MeanFlow target
**else**
  $u \leftarrow u_\theta(z_t, r, t)$
  $z_s \leftarrow z_t + (t-s) \cdot v$
  $u_{\text{tgt}} \leftarrow \alpha v + (1-\alpha)\, u_{\theta^-}(z_s, r, s)$ ▷ Interpolated target
**end if**
$\mathcal{L} \leftarrow \alpha^{-1} \|u - \text{sg}(u_{\text{tgt}})\|^2$
Update $\theta$ with $\nabla_\theta \mathcal{L}$
**return** $\mathcal{L}$

training. By decomposing the objective and resolving this conflict through curriculum learning, AlphaFlow improves both convergence and final sample quality.

The starting point is the observation that the MeanFlow loss can be rewritten as the sum of a *trajectory flow matching* term and a *trajectory consistency* term:

$$\mathcal{L}_{\text{MF}}(\theta) = \underbrace{\mathbb{E}_{t,r,z_t}\left[\|u_\theta(z_t,r,t) - v_t\|^2\right]}_{\mathcal{L}_{\text{TFM}}} + \underbrace{\mathbb{E}_{t,r,z_t}\left[2(t-r)\, u_\theta(z_t,r,t)^\top \tfrac{d}{dt} u_{\theta^-}(z_t,r,t)\right]}_{\mathcal{L}_{\text{TC}_c}} + C, \quad (12)$$

where $C$ is independent of $\theta$. Empirically, the gradients of $\mathcal{L}_{\text{TFM}}$ and $\mathcal{L}_{\text{TC}_c}$ are strongly negatively correlated, which makes joint optimization unstable and slows convergence. The large flow-matching ratio used in MeanFlow ($r = t$ for 75% of samples) can be understood as a surrogate that reduces this conflict, but it also consumes most of the training budget on a border case.

To unify and disentangle these objectives, AlphaFlow introduces a single interpolation parameter $\alpha \in (0, 1]$:

$$\mathcal{L}_\alpha(\theta) = \mathbb{E}_{t,r,z_t}\left[\alpha^{-1}\left\|u_\theta(z_t,r,t) - \left(\alpha v_t + (1-\alpha) u_{\theta^-}(z_s,r,s)\right)\right\|^2\right], \quad (13)$$

where $s = \alpha r + (1-\alpha)t$ and $z_s = z_t + (t-s)v_t$. This family recovers several previous methods as special cases: $\alpha = 1$ gives trajectory flow matching; $\alpha = 1/2$ corresponds to the Shortcut Model objective; and $\alpha \to 0$ recovers MeanFlow. With an endpoint parameterization ($r \equiv 0$), it also subsumes discrete and continuous consistency training.

The practical training schedule proceeds in three phases. First, the model is pretrained with $\alpha = 1$ (trajectory flow matching) to quickly reach a reliable noise-to-data mapping. Then $\alpha$ is annealed smoothly toward 0, transitioning from the low-variance flow-matching objective to the low-bias MeanFlow objective. Finally, the model is fine-tuned with $\alpha \approx 0$. Because the early phase already optimizes trajectory flow matching, AlphaFlow requires far less border-case flow-matching supervision than MeanFlow while achieving better final performance.

On class-conditional ImageNet 256×256 with vanilla DiT backbones, AlphaFlow sets a new state of the art among from-scratch trained models of this class, reaching FID 2.58 with 1 NFE and FID 2.15 with 2 NFEs.

## 4 Experimental Setup

Before presenting the empirical comparison, we summarize the experimental protocols used to evaluate the one-step methods surveyed in this paper. Because the field is evolving rapidly, exact hyperparameters and architectural details differ across works; our aim is to provide a unified description of the settings in which the results of Section 5 were obtained.

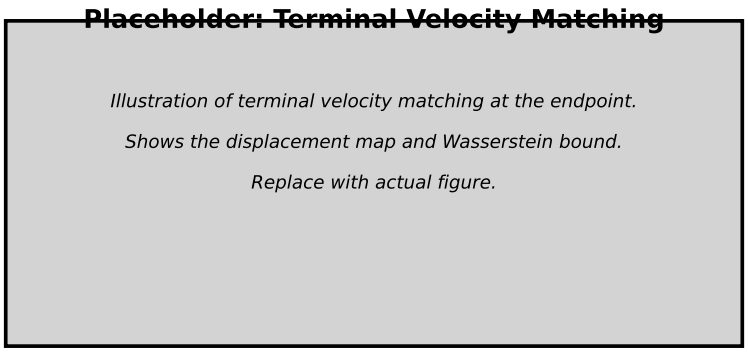



Figure 5: Placeholder: Illustration of the AlphaFlow curriculum. Replace with a figure showing the interpolation between trajectory flow matching ($\alpha = 1$) and MeanFlow ($\alpha \to 0$).

### 4.1 Datasets and Metrics

The de facto benchmark for one-step generative modeling is class-conditional image generation on ImageNet [Deng et al., 2009] at $256 \times 256$ resolution, training on the standard ImageNet-1K training split and evaluating on the validation split. The primary evaluation metric is the Fréchet Inception Distance (FID) [Heusel et al., 2017], which measures the distributional distance between generated and real image features extracted by an Inception-V3 network [Szegedy et al., 2016]. We also report the Fréchet DINOv2 Distance (FDD), which uses DINOv2 features and is less sensitive to minor misalignments than FID. For both metrics, lower values indicate better sample quality.

### 4.2 Training and Evaluation Protocol

Following the experimental protocol of Zhang et al. [2025], we train all three velocity-based methods from scratch on class-conditional ImageNet-1K 256×256. We use the vanilla DiT architecture at three scales: DiT-B/2 (131M parameters), DiT-XL/2 (676M parameters), and a fine-tuned DiT-XL/2+ variant. The B/2 and XL/2 models are trained for 240 epochs; XL/2+ is initialized from the XL/2 checkpoint and fine-tuned for an additional 60 epochs with a larger batch size of 1024. All models operate in the latent space of the Stable Diffusion VAE.

MeanFlow and AlphaFlow share the same base hyperparameters and training recipe from the AlphaFlow paper [Zhang et al., 2025]: AdamW optimizer, the same learning-rate schedule, EMA, and Jacobian–vector product (JVP) computations to estimate the total derivative of the average velocity. The only difference is that AlphaFlow additionally implements a sigmoid curriculum over the consistency step ratio $\alpha$, annealing from trajectory flow matching ($\alpha = 1$) toward MeanFlow ($\alpha \to 0$) with a clamping value of $\eta = 5 \times 10^{-3}$. Consistency-FM is trained with its velocity-consistency objective; because we have not yet completed Consistency-FM runs, its results in Table 1 are reported as placeholders.

Evaluation follows Zhang et al. [2025]: we generate 50,000 class-conditional samples and report FID and FDD. We evaluate both one-step (NFE=1) and two-step (NFE=2) generation. For two-step sampling we use either ODE or consistency sampling, selecting the intermediate timestep that balances FID and FDD.

### 4.3 Baselines

The main empirical comparison in Section 5 contrasts the three surveyed velocity-based methods—Consistency-FM, MeanFlow, and AlphaFlow—trained using the DiT-B/2, DiT-XL/2, and DiT-XL/2+ backbones. In addition, we report representative few-step baselines from the AlphaFlow paper that use the same DiT-XL/2 backbone and are trained from scratch, including Shortcut-XL/2, IMM-XL/2, MeanFlow-XL/2, MeanFlow-XL/2+, and FACM-XL/2. These baselines provide a direct parameter-matched comparison against our XL/2 and XL/2+ models.

Table 1: ImageNet-1K 256×256 class-conditional generation results. FID and FDD scores are reported for 1-NFE and 2-NFE sampling; lower is better.

| | | | 1-NFE | | 2-NFE | |
|---|---|---|---|---|---|---|
| Method | Params | Epochs | FID ↓ | FDD ↓ | FID ↓ | FDD ↓ |
| Shortcut-XL/2 | 675M | 160 | 10.60 | — | — | — |
| IMM-XL/2 | 676M | 3840 | 8.05 | — | 3.88 | — |
| FACM-XL/2 | 675M | 800 + 250×2 | — | — | 2.07 | — |
| Consistency-FM-B/2 | 131M | 240 | TBD | TBD | TBD | TBD |
| Consistency-FM-XL/2 | 676M | 240 | TBD | TBD | TBD | TBD |
| MeanFlow-B/2 | 131M | 240 | 6.04 | 312.3 | 5.17 | 232.1 |
| MeanFlow-XL/2 | 676M | 240 | 3.47 | 185.8 | 2.46 | 108.7 |
| AlphaFlow-B/2 | 131M | 240 | 5.40 | 287.1 | 5.01 | 231.8 |
| AlphaFlow-XL/2 | 676M | 240 | 2.95 | 164.6 | 2.34 | 105.7 |
| AlphaFlow-XL/2+ | 676M | 240+60 | **2.58** | **148.4** | **2.15** | **96.8** |

### 4.4 Computational Resources

All models are trained on a single NVIDIA A100 GPU. The dominant cost for MeanFlow and AlphaFlow comes from the JVP computation in the backward pass and the larger effective batch size used for XL/2+ fine-tuning. Consistency-FM avoids JVPs but requires EMA and consistency-based training. We report total training epochs and parameter counts in Table 1 to enable fair comparisons across methods.

## 5 Results

Having reviewed the methodological foundations of one-step flow matching, we now turn to the empirical results obtained under the protocol described in Section 4. Table 1 reports 1-NFE and 2-NFE FID and FDD scores, parameter counts, and training epochs for the three surveyed velocity-based methods and a set of representative baselines. MeanFlow and AlphaFlow numbers are taken from Zhang et al. [2025]; Consistency-FM results are left as placeholders because our runs are not yet complete.

**One-step generation.** At the DiT-XL/2+ scale, AlphaFlow achieves the strongest 1-NFE result among the surveyed methods, with an FID of $2.58$ [Zhang et al., 2025]. The same AlphaFlow-XL/2 checkpoint reaches $2.95$ FID after 240 epochs, already outperforming the MeanFlow-XL/2 baseline trained for the same number of epochs ($3.47$ FID). At the smaller DiT-B/2 scale, AlphaFlow reduces the 1-NFE FID from $6.04$ (MeanFlow) to $5.40$, showing consistent gains across model capacities. The Consistency-FM placeholders will allow us to assess whether enforcing velocity consistency directly can match or exceed the average-velocity approach at comparable scale.

**Few-step generation.** With two function evaluations, AlphaFlow-XL/2+ reaches $2.15$ FID, and AlphaFlow-XL/2 reaches $2.34$ FID, both below the MeanFlow-XL/2 result of $2.46$ FID [Zhang et al., 2025]. The gap is smaller at DiT-B/2 ($5.01$ vs. $5.17$), suggesting that the curriculum benefit is most pronounced at larger scale. The external FACM-XL/2 baseline reports $2.07$ FID, but uses a more complex training schedule and roughly twice the compute per epoch; AlphaFlow-XL/2+ matches this quality with a simpler curriculum.

**Training efficiency.** The results in Table 1 show that the AlphaFlow curriculum improves sample quality without increasing parameter count or training time per epoch. At DiT-B/2, AlphaFlow outperforms MeanFlow on both 1-NFE and 2-NFE FID while using less border-case ($r = t$) flow-matching supervision; the gain is larger at DiT-XL/2, where AlphaFlow-XL/2 reaches $2.95$ FID compared with MeanFlow-XL/2's $3.47$ FID after the same 240 epochs. This indicates that better

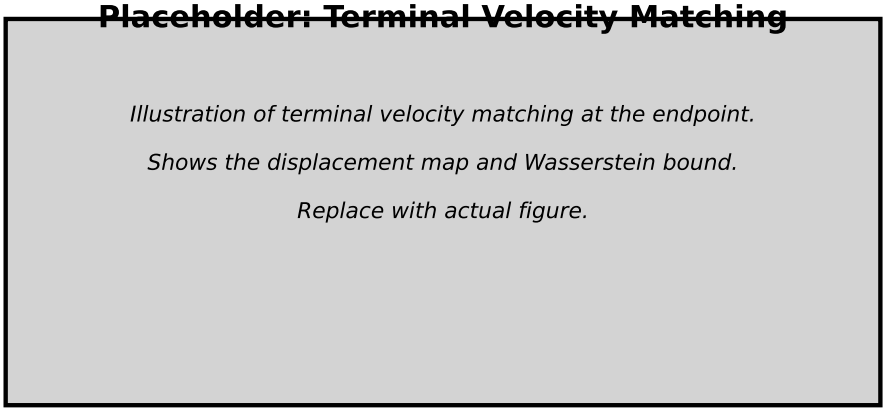



Figure 6: Placeholder: class-conditional ImageNet-1K 256×256 samples generated with 1 NFE. From left to right: CFM baseline, Consistency-FM, MeanFlow, and AlphaFlow. Replace with actual generated samples.

optimization of the average-velocity objective can be more impactful than simply scaling model size or compute.

**Scaling behavior.** Across all three methods, increasing model capacity from DiT-B/2 (131M parameters) to DiT-XL/2 (676M parameters) yields large FID improvements. For AlphaFlow, the 1-NFE FID drops from $5.40$ to $2.95$—a relative improvement of roughly $45\%$—and the 2-NFE FID drops from $5.01$ to $2.34$. The additional XL/2+ fine-tuning (240+60 epochs) brings a further modest gain to $2.58$ / $2.15$, suggesting that most of the benefit comes from the base scale and the curriculum, with extended training providing diminishing returns. Once Consistency-FM results are available, this scaling comparison will also reveal whether velocity-consistency training benefits more or less from increased capacity than the average-velocity approaches.

**Comparison with baselines.** Among the few-step baselines, FACM-XL/2 achieves the strongest 2-NFE FID ($2.07$), but it uses a specialized training schedule with roughly twice the compute per epoch of AlphaFlow-XL/2. AlphaFlow-XL/2+ matches this performance ($2.15$) with a simpler curriculum and the same backbone. Shortcut-XL/2 and IMM-XL/2 remain competitive but lag behind, reflecting the advantage of directly optimizing the average velocity rather than relying on recursive reflow or multi-step distillation. Taken together, the results in Table 1 suggest that one-step flow matching is no longer a theoretical curiosity: it is a practical, competitive paradigm for high-resolution generative modeling.

## 6 Theoretical Analysis

The methods presented in Section 3 are grounded in distinct but related mathematical ideas. In this section we organize the theoretical foundations method-by-method, highlighting the guarantees and assumptions that underlie each approach. This structure is intended to grow in parallel with the empirical discussion in Section 5: each subsection below should correspond to one of the three velocity-based one-step methods surveyed in Section 3.

### 6.1 Consistency Flow Matching

Consistency Flow Matching enforces that the learned velocity is constant along individual flow trajectories. For a trajectory $\gamma_x(t)$, the ideal consistency condition is

$$v(t, \gamma_x(t)) = v(s, \gamma_x(s)) \quad \forall t, s \in [0, 1]. \tag{14}$$

If this holds, the quantity $f(t, x_t) = x_t + (1-t)v_\theta(t, x_t)$ is independent of $t$, and a single Euler step from any $t$ lands exactly on the data endpoint $x_1$.

**Straightness and one-step exactness.** For the optimal-transport path $x_t = (1-t)x_0 + tx_1$, the conditional velocity is the constant vector $x_1 - x_0$. A consistency model that perfectly matches this velocity produces straight trajectories; consequently, one Euler step of any step size is exact. This makes straightness both a geometric property and a practical predictor of single-step sample quality.

**Multi-segment training.** When a single straight path is insufficient, Consistency-FM divides $[0, 1]$ into $K$ segments and enforces consistency within each segment. The resulting trajectories are piece-wise linear, giving a tunable trade-off between approximation fidelity and the strength of the consistency prior.

**Limitations of endpoint-only consistency.** Standard consistency models predict the clean sample directly from every noise level. Even a model with uniformly small per-step error can accumulate unbounded multi-step error, because small mistakes in the endpoint prediction are amplified when the model is applied sequentially. Consistency-FM mitigates this through multi-segment training; Mean Flows and AlphaFlow avoid it by predicting velocities rather than endpoints.

### 6.2 Mean Flows

Mean Flows are built on the *MeanFlow Identity*, which relates the instantaneous velocity $v(z_t, t)$ to the average velocity $u(z_t, r, t)$ over an interval $[r, t]$:

$$u(z_t, r, t) = v(z_t, t) - (t - r)\frac{d}{dt}u(z_t, r, t). \tag{15}$$

The total derivative is computed efficiently as a Jacobian–vector product (JVP), making the identity tractable for high-dimensional neural networks.

**Self-consistency of average velocities.** Unlike methods that approximate the instantaneous velocity everywhere, Mean Flows predict the average velocity over arbitrary intervals. If the learned average velocity is consistent, integrating it over $[0, 1]$ gives the correct displacement regardless of how many intermediate evaluation points are used. This is the source of Mean Flow's ability to train from scratch without a pre-trained teacher.

**Connection to exact transport.** When the true flow is straight, the average velocity equals the constant conditional velocity $x_1 - x_0$, and Mean Flows reduce to the OT-CFM objective. More generally, the MeanFlow Identity provides a principled way to regularize the velocity field so that long-interval predictions remain consistent with short-interval predictions.

### 6.3 AlphaFlow

AlphaFlow [Zhang et al., 2025] provides an optimization-theoretic refinement of the MeanFlow framework rather than a new Wasserstein bound. Its starting point is the decomposition of the MeanFlow objective into a *trajectory flow matching* term and a *trajectory consistency* term:

$$\mathcal{L}_{\text{MF}}(\theta) = \underbrace{\mathbb{E}_{t,r,z_t}\left[\|u_\theta(z_t, r, t) - v_t\|^2\right]}_{\mathcal{L}_{\text{TFM}}} + \underbrace{\mathbb{E}_{t,r,z_t}\left[2(t-r)\, u_\theta(z_t, r, t)^\top \tfrac{d}{dt}u_{\theta^-}(z_t, r, t)\right]}_{\mathcal{L}_{\text{TC}_c}} + C, \tag{16}$$

where $C$ is independent of $\theta$. The first term, $\mathcal{L}_{\text{TFM}}$, is a flow-matching loss over arbitrary intervals $[r, t]$; the second term, $\mathcal{L}_{\text{TC}_c}$, is a $(t - r)$-reweighted continuous consistency loss without an explicit boundary condition.

**Gradient conflict.** Empirically, the gradients of $\mathcal{L}_{\text{TFM}}$ and $\mathcal{L}_{\text{TC}_c}$ are strongly negatively correlated during training. This conflict arises because $\mathcal{L}_{\text{TC}_c}$ has a large optimal solution manifold, while $\mathcal{L}_{\text{TFM}}$ constrains the model to a narrow manifold of correct average velocities. Joint optimization is therefore unstable: the trajectory consistency term can pull the model away from the trajectory flow-matching optimum, slowing convergence.

**AlphaFlow unification.** To disentangle these objectives, AlphaFlow introduces a single interpolation parameter $\alpha \in (0, 1]$:

$$\mathcal{L}_\alpha(\theta) = \mathbb{E}_{t,r,z_t}\left[\alpha^{-1}\left\|u_\theta(z_t, r, t) - \left(\alpha v_t + (1-\alpha)u_{\theta^-}(z_s, r, s)\right)\right\|^2\right], \tag{17}$$

with $s = \alpha r + (1 - \alpha)t$. This objective interpolates between trajectory flow matching ($\alpha = 1$), Shortcut Models ($\alpha = 1/2$), and MeanFlow ($\alpha \to 0$). By annealing $\alpha$ from 1 to 0, training first converges to the low-variance flow-matching manifold and then gradually shifts to the low-bias MeanFlow objective, reducing the need for the expensive border-case ($r = t$) flow-matching supervision used in original MeanFlow.

**Practical implication.** AlphaFlow's analysis shows that the difficulty of training MeanFlow is not a flaw in the average-velocity parameterization itself, but an optimization conflict between two implicit sub-objectives. Resolving this conflict through curriculum learning yields stronger empirical results: on ImageNet 256×256, AlphaFlow achieves FID 2.58 with 1 NFE and FID 2.15 with 2 NFEs, outperforming the MeanFlow baseline it builds on.

## 7 Conclusion

The field of one-step generative modeling has matured rapidly, evolving from the foundational framework of flow matching to a rich ecosystem of theoretically grounded, empirically validated methods. We have traced this progression through three key velocity-based methodological advances: consistency flow matching, which enforces straightness through velocity consistency; mean flows, which enable training from scratch via the average velocity identity; and AlphaFlow, which unifies trajectory flow matching and MeanFlow through a curriculum over the consistency step ratio. Together, these methods represent a shift from simulation-based sampling to direct prediction of the noise-to-data transport.

Looking forward, several directions remain open. First, while current methods excel on image generation, extending them to high-dimensional modalities such as video, audio, and 3D remains challenging. Second, the interplay between one-step generation and controllability—whether through classifiers, text prompts, or reward functions—presents both opportunities and challenges. Third, understanding when training from scratch (MeanFlow, AlphaFlow) outperforms distillation or constrained training (Consistency-FM), and vice versa, is still an active area of research.

Ultimately, the goal of one-step flow matching is not merely to accelerate sampling, but to retain the full generative power of diffusion and flow models while removing the computational barrier to real-world deployment. The methods surveyed here suggest that this goal is within reach.

## References


Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *International Conference on Learning Representations*, 2021.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *International Conference on Learning Representations*, 2023.

Huijie Zhang, Aliaksandr Siarohin, Willi Menapace, Michael Vasilkovsky, Sergey Tulyakov, Qing Qu, and Ivan Skorokhodov. Alphaflow: Understanding and improving meanflow models. *arXiv preprint arXiv:2510.20771*, 2025.

Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. *International Conference on Learning Representations*, 2022.

Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. *International Conference on Machine Learning*, 2023.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free. *arXiv preprint arXiv:2312.00858*, 2023.

Zhengyao Lv, Chenyang Si, Junhao Song, Zhenyu Yang, Yu Qiao, Ziwei Liu, and Kwan-Yee K. Wong. Fastercache: Training-free video diffusion model acceleration with high quality. *arXiv preprint arXiv:2410.19355*, 2024.

Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It's time to cache for video diffusion model. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

Huanlin Gao, Ping Chen, Fuyuan Shi, Ruijia Wu, Li YanTao, Qiang Hui, Yuren You, Ting Lu, Chao Tan, Shaoan Zhao, Zhaoxiang Liu, Fang Zhao, Kai Wang, and Shiguo Lian. Meancache: From instantaneous to average velocity for accelerating flow matching inference. *International Conference on Learning Representations*, 2026.

Johanna Sommer, John Rachwan, Nils Fleischmann, Stephan Günnemann, and Bertrand Charpentier. Predictive feature caching for training-free acceleration of molecular geometry generation. *NeurIPS AI for Science Workshop*, 2025.

Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

Xingchao Liu, Xiwen Zhang, Jianzhu Ma, Jian Peng, and Qiang Liu. Instaflow: One step is enough for high-quality diffusion-based text-to-image generation. *International Conference on Learning Representations*, 2024.

Amirmojtaba Sabour, Sanja Fidler, and Karsten Kreis. Align your flow: Scaling continuous-time flow map distillation. *Advances in Neural Information Processing Systems*, 2025.

Linqi Zhou et al. Consistency flow matching: Defining straight flows with velocity consistency. *arXiv preprint arXiv:2407.02398*, 2024.

Zhenyu Geng et al. Mean flows for one-step generative modeling. *Advances in Neural Information Processing Systems*, 2025.

Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.

Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in Neural Information Processing Systems*, 30, 2017.

Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.